

CISC 667

Introduction to Autonomous Driving

Lecture 8
Nov 12, 2025

Today's Outline

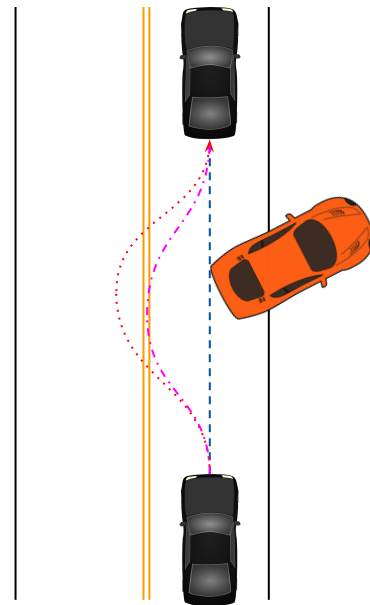
- Planning Algorithms
 - Local planning, trajectory generation and optimization
- Decision Making
 - Markov Decision Process
- Control
 - PID
 - MPC
 - Pure Pursuit

Local Planning

Input: Reference lane/route, ego state, predictions, road limits.

Output: Time-parameterized trajectory $(x(t), y(t), v(t), a(t))$ over **3–6 s** that is **collision-free**, **feasible**, and **smooth**.

Loop rate: ~10–20 Hz (re-plan continuously).



Frenet-based Motion Planning

Input:

- Lane/corridor centerline $\mathbf{r}(s)$ with bounds, speed limits
- Ego state $x, y, \psi, v, a, \dot{\psi}, \dot{v}, \dot{a}$ (optionally yaw rate)
- Predicted actors (time-stamped footprints)
- Vehicle limits on $v, a, \dot{v}, \dot{a}$, jerk, $\delta, \dot{\delta}, \ddot{\delta}$

Output:

- Trajectory $(x(t), y(t), v(t), a(t))$, horizon **3–6 s** — **collision-free, feasible, smooth**

Loop rate: ~10–20 Hz

1. Project ego $\rightarrow$ Frenet (s_0, d_0)
2. Sample lateral $(d_T, \dot{d}_T, \ddot{d}_T)$ and longitudinal $s_T, \dot{s}_T, \ddot{s}_T$
3. Fit polynomials: $d(t)$ **quintic**, $s(t)$ **quartic/quintic**.
4. Map to (x, y) , check limits & collisions, **score and select best**.

Polynomial-based trajectory generation

Input (polynomial-based, frame-agnostic):

- Current state at $t=0$: position/heading, v_0, a_0 in **Cartesian, Frenet, or body frame**.
- Target state at $t=T$ (or small set): desired position/speed (optional a_T); candidate durations $T \in [3, 6]$ s.
- **Drivable corridor/bounds** (lane centerline optional), obstacle **predictions**.
- Vehicle limits: max $v, a, \dot{a}$, jerk, curvature, steering δ , steering rate $\dot{\delta}$.

Output:

- Time-parameterized trajectory $(x(t), y(t), v(t), a(t))$ over **3–6 s, collision-free, feasible, smooth** (with discrete waypoints for the controller).

Loop rate (~10–20 Hz):

- Sample goals & TTT → **fit quintic polynomials** to meet start/end states (per axis or $s/ds/ds/d$) → convert to Cartesian if needed → **check bounds/limits/collisions** → **score & select** best → warm-start next cycle.

Trajectory optimization as a nonlinear program

Trajectory optimization is a calculator that, for the next few seconds, **picks the best steering and speed** that satisfy those rules **simultaneously**.

It **bakes in hard limits** (speed, accel, steering, clearance) so the plan is **physically feasible and safe**.

It **balances trade-offs** (progress vs. comfort vs. safety) explicitly with weights, instead of guessing.

In **tight, changing scenes** (merges, obstacles moving), it adapts each cycle and gives a **best-available plan**—or a safe fallback if none fits.

Markov Decision Process

What it is: A decision framework with **states, actions, transition probabilities, and per-step rewards/costs** (not just a Markov chain).

Why not a Markov chain: Chains have **no actions or objective**; MDPs choose actions to **optimize long-term reward**.

Immediate reward (stage cost): Encodes trade-offs each step—**progress/time**, **safety** (collision/clearance/TTC), **comfort** (jerk/steer-rate), **rules** (speed/lights), **efficiency** (energy).

Objective: Maximize $\sum_t r_t$ (or minimize $\sum_t \ell_t$) over the horizon; add a **terminal** term for end goals; discount optional.

Where used: Behavior/decision layer (gap acceptance, merge/yield). Local planning often uses an **equivalent cost formulation** (MPC/NLP).

Reality: Often a **POMDP** due to partial observability; still optimized via value/policy methods or model-predictive surrogates.

A **POMDP** is an MDP with **hidden state**: you maintain a **belief** (probability over states) from **noisy observations** and choose actions that **maximize expected reward** under that belief.

PID Control

- **Integral (I):** It will **drive steady-state error to zero** (even if K_p is small or there's a constant disturbance like a hill). But it does so **slowly** and can cause **overshoot/oscillation** if K_i is high or without **anti-windup**.
- **Proportional (P):** Larger $K_p \rightarrow$ **faster** response but **more overshoot**/risk of oscillation. Too small K_p leaves a **bias** that I must clean up.
- **Derivative (D):** Adds **damping** (reacts to error slope), cutting overshoot/oscillation; keep it modest and filtered.
- <https://www.luisllamas.es/en/pid-controller-simulator/>

PID Control in Terms of AV

Longitudinal (speed/throttle–brake)

- **P:** Pushes proportional to speed error → quick correction.
- **I:** Cancels steady bias (hills/drag) → zero offset.
- **D:** Damps overshoot on setpoint/grade changes.

Lateral (steering/path tracking)

- **P:** Steers into cross-track/heading error.
- **I:** Removes constant lane offset (crosswind/camber).
- **D:** Damps oscillation/weaving.

MPC Control

What: Optimization-based controller that, each cycle, predicts motion over a short horizon and finds the **control sequence** minimizing a cost **subject to constraints**.

How (receding horizon): Predict → optimize → apply first input → repeat.

Cost (typical): tracking error + control effort + rate/jerk (optionally progress).

Constraints: speed/accel/steering/lat-acc (and sometimes clearance via soft margins).

Why use it: **Anticipates** upcoming curvature/slowdowns, **handles constraints natively**, coordinates multiple inputs.

Trade-off: Needs compute and a decent model; tuning/scaling matter.

Pure Pursuit

What: Geometric **preview** controller that, each cycle, steers toward a **lookahead point** on the path (no optimization).

How (preview loop): Pick lookahead L_d → compute heading α to that point → curvature $k = \frac{2 \sin \alpha}{L_d}$ → steering $\delta = \arctan(L_d k)$ → apply → repeat.

Inputs: Path waypoints, ego pose (x, y, ψ) , wheelbase L , **lookahead law** $L_d(v)$ (e.g., $k_1 + k_2 v$).

Why use it: **Simple, fast, robust** path tracking with minimal tuning; good baseline for lanes/waypoints.

Constraints: None explicit (no curvature/lat-acc/steer limits); speed handled by a separate longitudinal loop.

Trade-off: Can **corner-cut** and show **steady offset** on tight curves; sensitive to L_d (too small → oscillation, too large → sluggish).

Exercise

- Modify the existing onboard code so that you vehicle follows you in the corridor without hitting you when you walk and run.
- controller code: `src/pure_pursuit_controller/src/pure_pursuit_controller.cpp`
- vision models: `src/perception/models`
- perception code: `src/perception/perception/perception_node.py`
- control center code: `src/control_center/src/control_center_node.cpp`